Lecture 9: Latent variable models beyond Gaussian mixtures, topic models, language models

TTIC 31220: Unsupervised Learning and Data Analysis

Instructor: Karen Livescu

TTI-Chicago

April 20, 2026

Announcements

- Quiz 1 grades available on canvas, and on paper from me after class
- (Please come to office hours to discuss anything that gave you trouble on the quiz!)
- KL's office hour this week: Friday 10am
- HW2 due this Thursday
- Quiz 2 Monday 4/27
- Project guidelines available on canvas (under the project "Module"; some additional resources may be added)
  - This week: Come talk to us in office hours about project topics!
  - Need help finding partners?

Latent variable models beyond GMs: Graphical models

Graphical models

- Graphical language for representing and reasoning about random variables and their distributions
- Probability distributions (for a *single* random variable) $p(X)$
  discrete $X \Rightarrow$ PMF
  continuous $X \Rightarrow$ density

Statistical independence

- Definition and implications of independence:

$$\boxed{X \perp Y} \iff p(x \mid y) = p(x)$$

$$\Updownarrow \qquad\qquad \Updownarrow$$

$$p(x, y) = p(x)\,p(y) \iff p(y \mid x) = p(y)$$

Statistical independence

- Definition and implications of independence:

$$\boxed{X \perp Y} \iff p(x \mid y) = p(x)$$

$$\Updownarrow \qquad\qquad \Updownarrow$$

$$p(x, y) = p(x)\,p(y) \iff p(y \mid x) = p(y)$$

- Conditional independence:

$$\boxed{X \perp Y \mid Z} \iff p(x \mid y, z) = p(x \mid z)$$

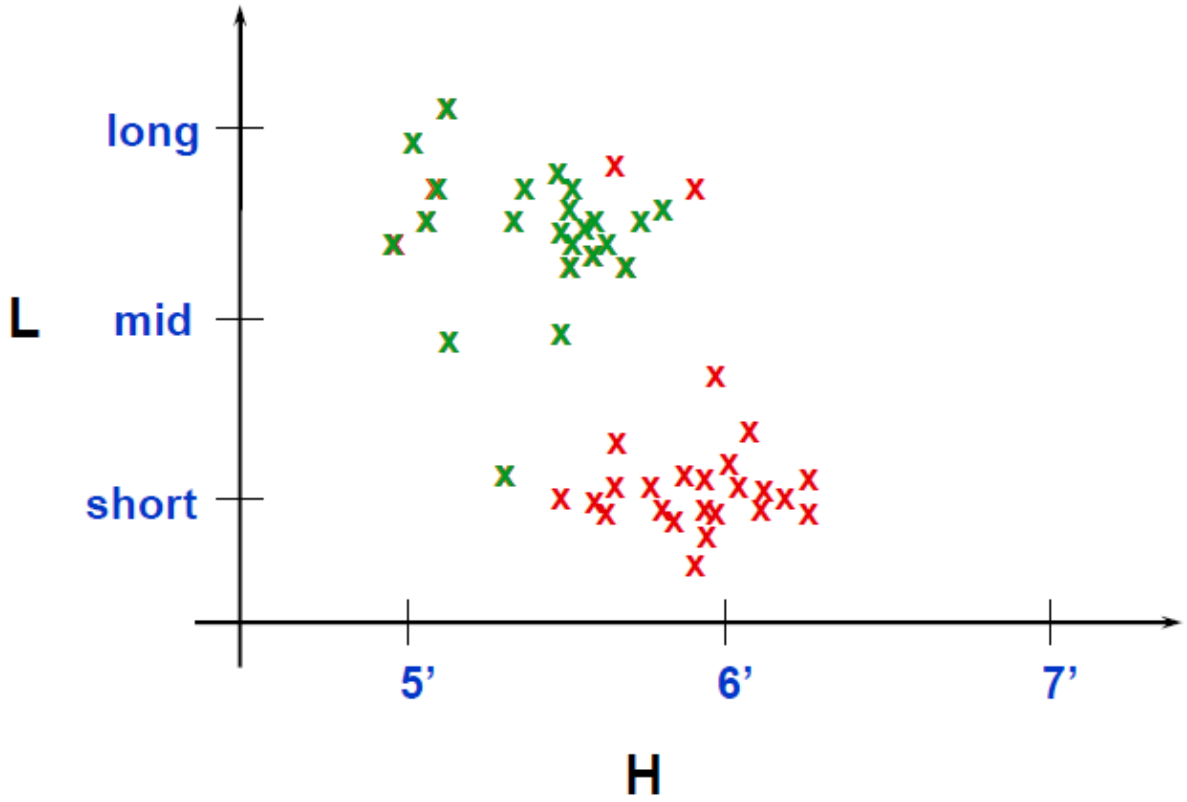
$$\Updownarrow \qquad\qquad \Updownarrow$$

$$p(x, y \mid z) = p(x \mid z)\,p(y \mid z) \iff p(y \mid x, z) = p(y \mid z)$$

Is hair length independent of height?

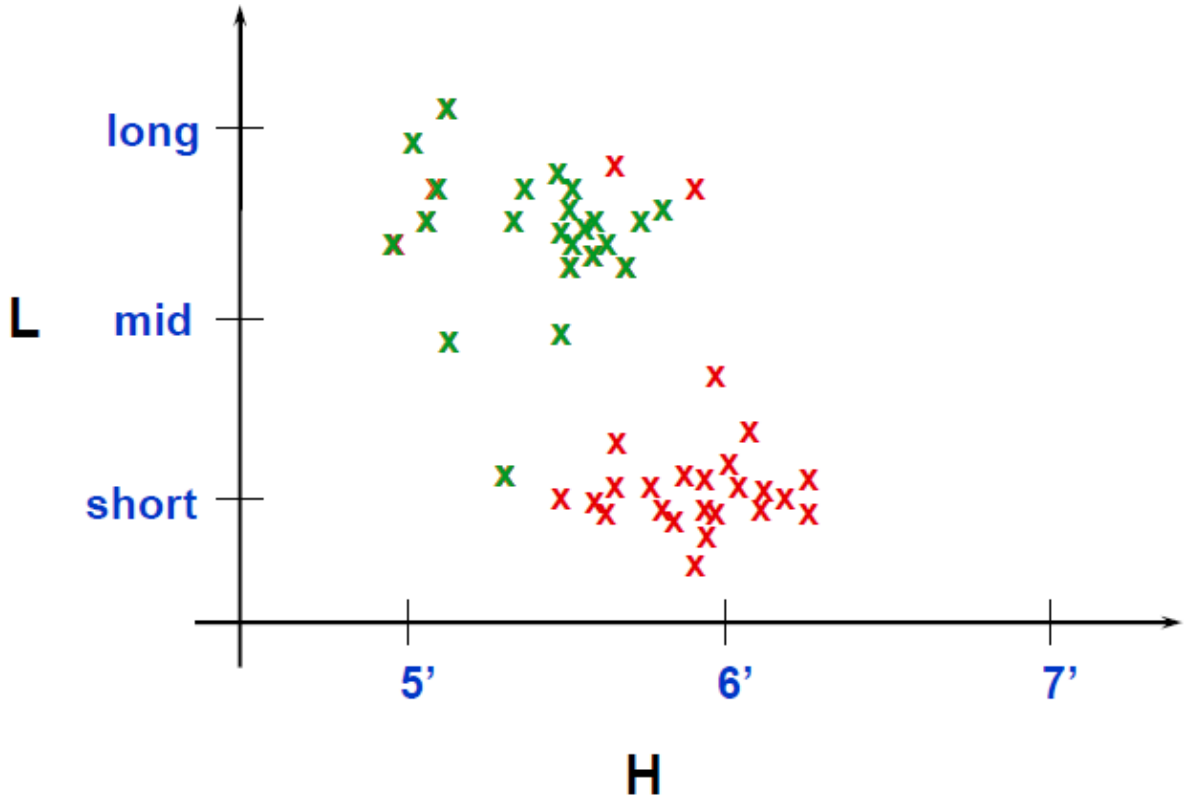
Is hair length independent of height?

- Measurements from a group of people:



Is hair length independent of height?

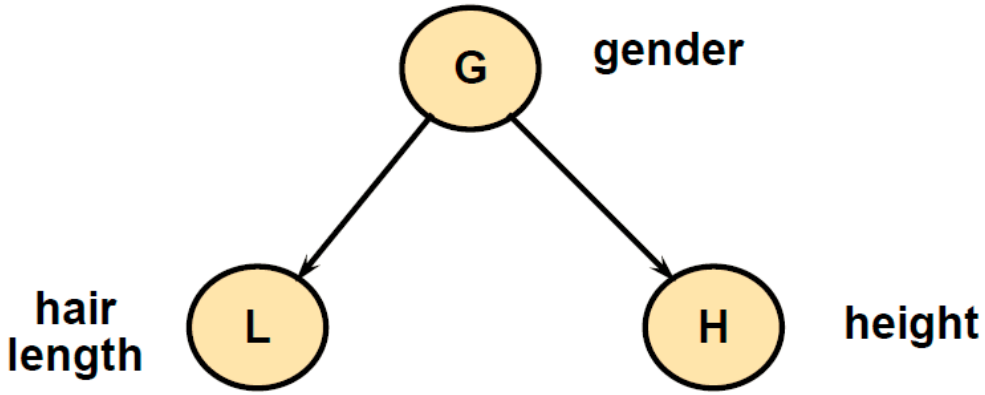
- Measurements from a group of people:



- Not in general; yes if we know the gender

Hair length and height

- We can draw it as a graph:

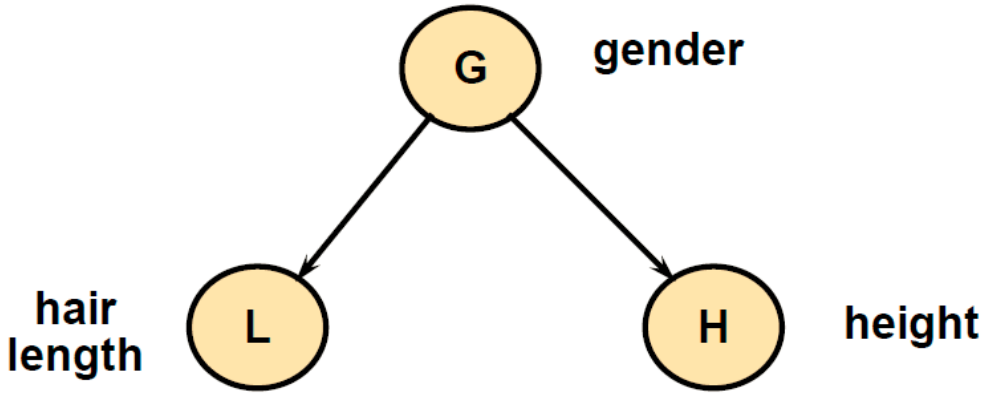


$$p(h \mid l) \neq p(h) \qquad H \not\perp L$$

$$p(h \mid l, g) = p(h \mid g) \qquad H \perp L \mid G$$

Hair length and height

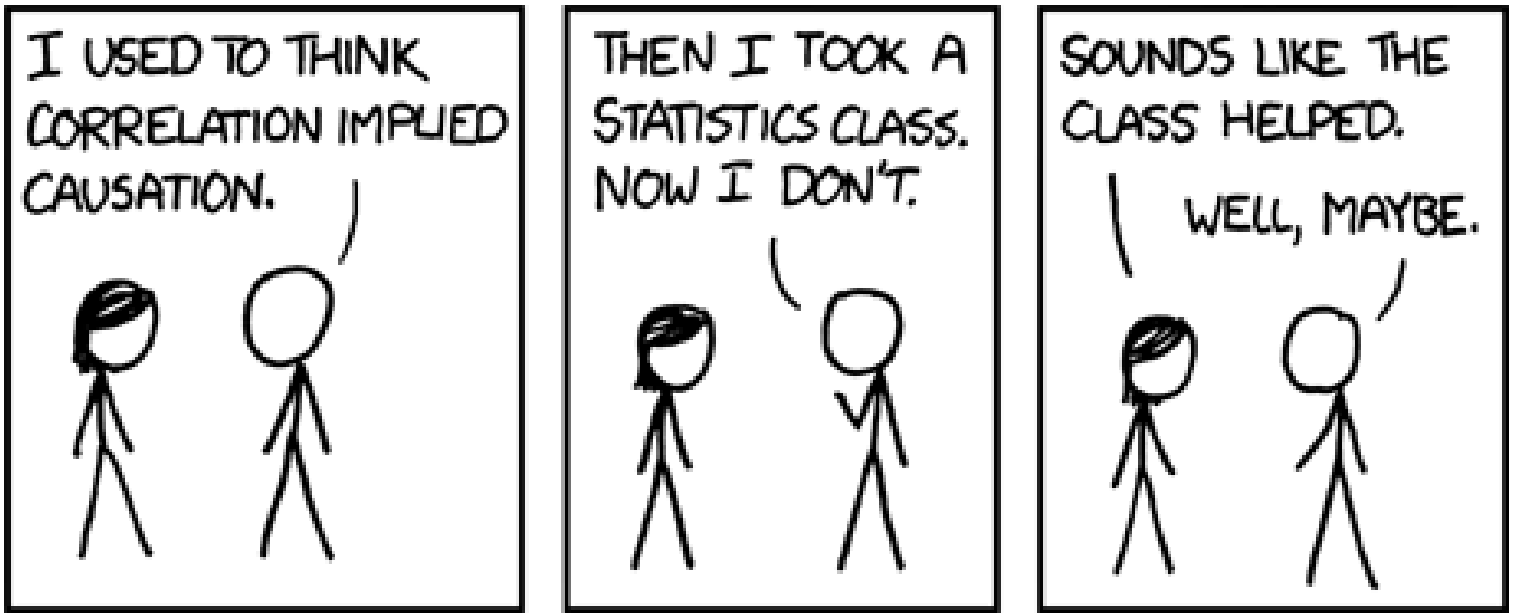
- We can draw it as a graph:



$$p(h \mid l) \neq p(h) \qquad H \not\perp L$$
$$p(h \mid l, g) = p(h \mid g) \qquad H \perp L \mid G$$

- This is the *common cause* situation: the latent variable G affects both H and L;
  H "affects" L only through G
- Note: "affects" $\neq$ "causes"

Correlation vs. causation



source: XKCD

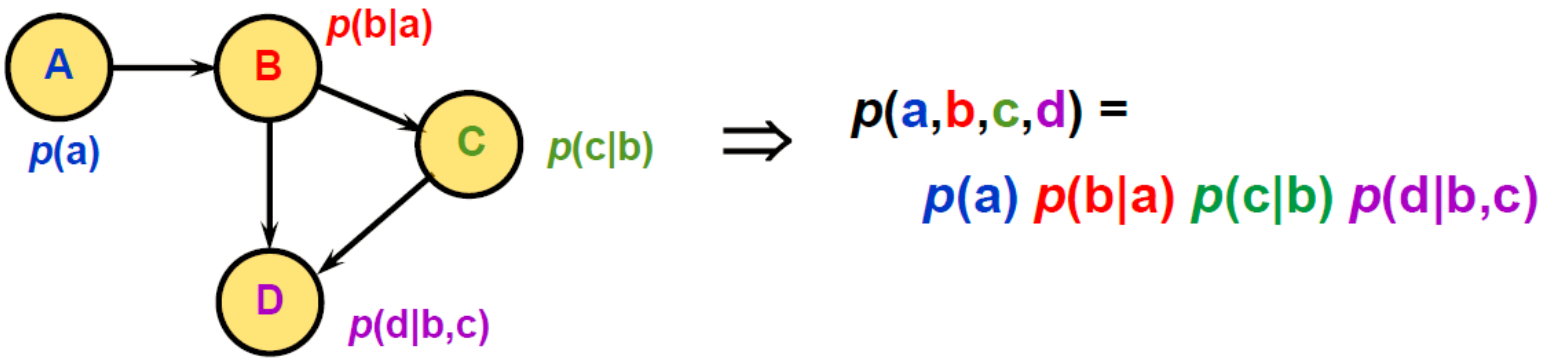
Note: Correlation vs. statistical dependence

- (I don't have a comic for this, but it's equally important)
- X, Y independent $\implies$ X, Y uncorrelated ($\rho_{X,Y} = 0$)
- The converse does not hold
- Exception: For jointly Gaussian variables, uncorrelatedness and independence are equivalent

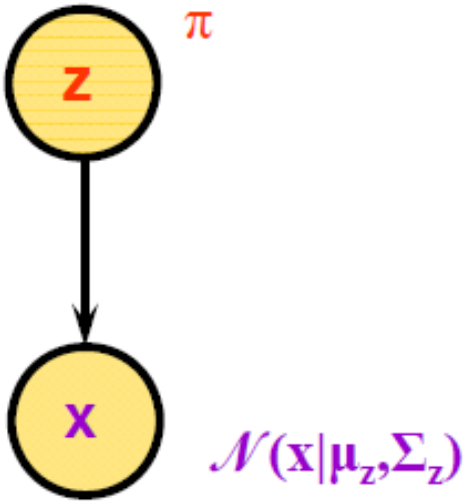
Bayesian networks

- Directed graphical model: a graph over random variables $X_1, \ldots, X_n$
- Must be a directed acyclic graph (DAG)
- Node (vertex) X_i with parents $\pi(X_i)$: local probability function $p\left(X_i|\pi(X_i)\right)$
- Joint probability:

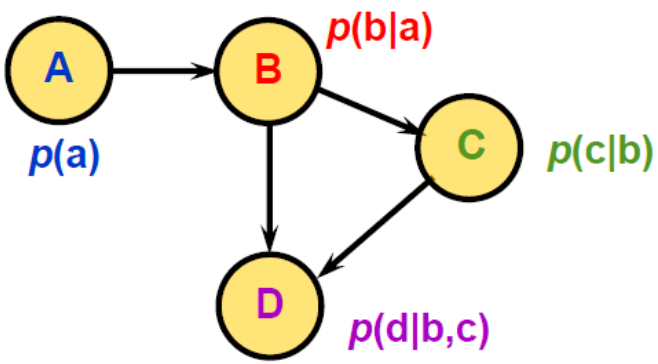
$$p(X_1, \ldots, X_n) = \prod_{i=1}^{n} p\left(X_i|\pi(X_i)\right)$$



Bayesian network for Gaussian mixture



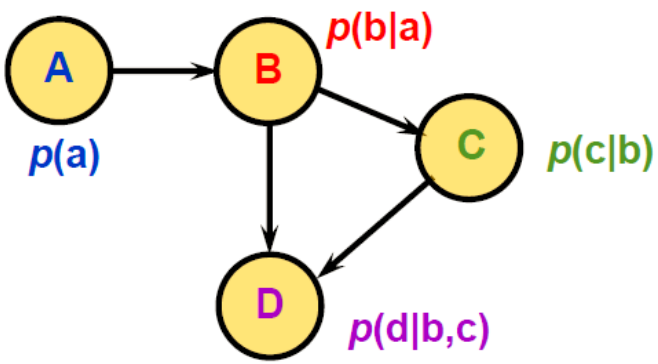
Factorization in BN



$\Rightarrow$ $p(a,b,c,d) = p(a)\,p(b|a)\,p(c|b)\,p(d|b,c)$

- Suppose a, b, c, d are discrete and each have 10 values
- Representing $p(a)$ requires 10 numbers (9, really)
- Representing $p(b|a)$ requires

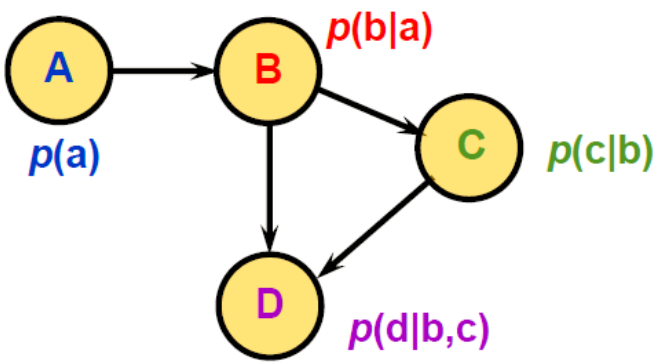
Factorization in BN



$\Rightarrow$ $p(a,b,c,d) = p(a)\,p(b|a)\,p(c|b)\,p(d|b,c)$

- Suppose a, b, c, d are discrete and each have 10 values
- Representing $p(a)$ requires 10 numbers (9, really)
- Representing $p(b|a)$ requires 100 numbers

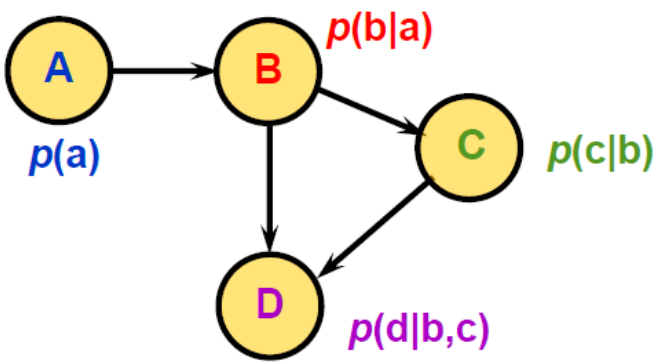
Factorization in BN



$\Rightarrow$ $p(a,b,c,d) = p(a)\,p(b|a)\,p(c|b)\,p(d|b,c)$

- Suppose a, b, c, d are discrete and each have 10 values
- Representing $p(a)$ requires 10 numbers (9, really)
- Representing $p(b|a)$ requires 100 numbers
- Without factorization (no BN): $p(a, b, c, d)$

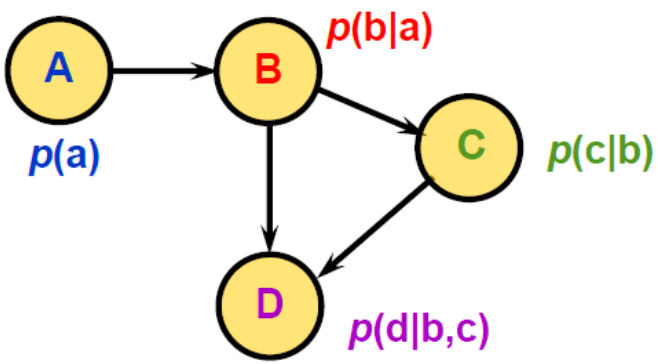
Factorization in BN



$\Rightarrow$ $p(a,b,c,d) = p(a)\,p(b|a)\,p(c|b)\,p(d|b,c)$

- Suppose a, b, c, d are discrete and each have 10 values
- Representing $p(a)$ requires 10 numbers (9, really)
- Representing $p(b|a)$ requires 100 numbers
- Without factorization (no BN): $p(a, b, c, d)$ 10,000
- With factorization:

Factorization in BN



$\Rightarrow$ $p(a,b,c,d) = p(a)\,p(b|a)\,p(c|b)\,p(d|b,c)$

- Suppose a, b, c, d are discrete and each have 10 values
- Representing $p(a)$ requires 10 numbers (9, really)
- Representing $p(b|a)$ requires 100 numbers
- Without factorization (no BN): $p(a, b, c, d)$ 10,000
- With factorization: only $10 + 100 + 100 + 1000 = 1{,}210$

Generative models of (text) documents

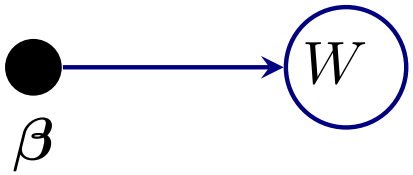
Topic models: Generative "mixture" models for documents

- Simple model for documents: words drawn i.i.d. from a distribution
- Graphical model: a generative process for a set of words



Topic models: Generative "mixture" models for documents

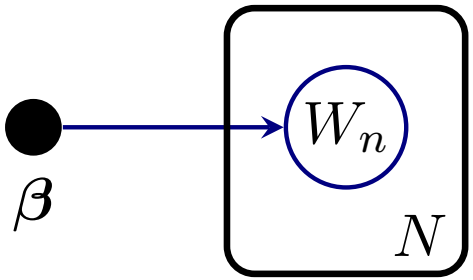
- Simple model for documents: words drawn i.i.d. from a distribution
- Graphical model: a generative process for a set of words



- Word distribution: $\boldsymbol{\beta}$ parameterizes distribution over words
  suppose we have dictionary of size M; then $\boldsymbol{\beta} \in [0,1]^M$, $\sum_m \beta_m = 1$.

Topic models: Generative "mixture" models for documents

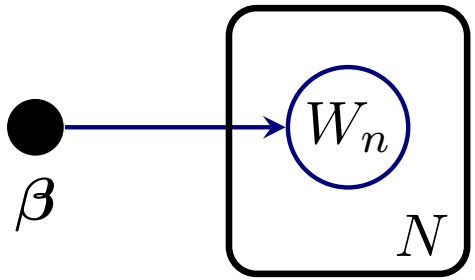
- Simple model for documents: words drawn i.i.d. from a distribution
- Graphical model: a generative process for a set of words



- Word distribution: $\boldsymbol{\beta}$ parameterizes distribution over words
  suppose we have dictionary of size M; then $\boldsymbol{\beta} \in [0,1]^M$, $\sum_m \beta_m = 1$.
- Plate notation: box ("plate") labeled N indicates N copies of the variables inside the box

Topic models: Generative "mixture" models for documents

- Simple model for documents: words drawn i.i.d. from a distribution
- Graphical model: a generative process for a set of words



- Word distribution: $\boldsymbol{\beta}$ parameterizes distribution over words suppose we have dictionary of size M; then $\boldsymbol{\beta} \in [0,1]^M$, $\sum_m \beta_m = 1$.
- Plate notation: box ("plate") labeled N indicates N copies of the variables inside the box
- Suppose the words are all in an article about sports. Then, $p(ball)$ =high; $p(field)$ =high; $p(baseball)$ =high; $p(eggplant)$=low.

Using generative models

- Generative model explains the observed process:

$$p(W_1, \ldots, W_N) = \prod_{i=1}^{N} p(W_i; \boldsymbol{\beta})$$

- Two main tasks:
  Inference: calculate probability of a set of words given the model.
  Learning: estimate model parameters from observed data.
- Simple approach to estimation: maximum likelihood
  given observed $W_1, \ldots, W_N$, estimate for each dictionary word m

$$\widehat{\beta}_m = \frac{\#(W_i = m)}{N} \qquad \text{where } \#(X) = \text{count of X}$$

Then:

$$\widehat{\boldsymbol{\beta}} = \operatorname*{argmax}_{\boldsymbol{\beta}} p(W_1, \ldots, W_N; \boldsymbol{\beta})$$

Discriminative use of generative models

- Suppose we have two topics, each with its own model $\boldsymbol{\beta}_1$, $\boldsymbol{\beta}_2$
- Given the document, want to classify it into one of the topics
- We should think about *class-conditional* distributions:

$$p(W_1, \ldots, W_N | c) = p(W_1, \ldots, W_N; \boldsymbol{\beta}_c)$$

- Simple classifier: predict class 1 if

$$p(W_1, \ldots, W_N | 1) \geq p(W_1, \ldots, W_N | 2)$$

Discriminative use of generative models

- Suppose we have two topics, each with its own model $\boldsymbol{\beta}_1$, $\boldsymbol{\beta}_2$
- Given the document, want to classify it into one of the topics
- We should think about *class-conditional* distributions:

$$p(W_1, \ldots, W_N | c) = p(W_1, \ldots, W_N; \boldsymbol{\beta}_c)$$

- Simple classifier: predict class 1 if

$$p(W_1, \ldots, W_N | 1) \geq p(W_1, \ldots, W_N | 2)$$

- This implicitly assumes $p(1) = p(2)$. More generally, we have

$$p(W_1, \ldots, W_N | 1)p(1) \geq p(W_1, \ldots, W_N | 2)p(2)$$

Where do topics come from?

- We can further assume generative model for the topics themselves
- Need a distribution over distributions!

Where do topics come from?

- We can further assume generative model for the topics themselves
- Need a distribution over distributions!
- The Dirichlet distribution

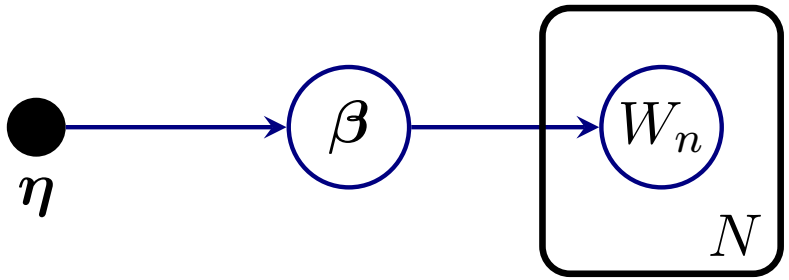
$$p(\boldsymbol{\beta}; \boldsymbol{\eta}) = \frac{1}{B(\boldsymbol{\eta})} \prod_{m=1}^{M} \beta_m^{\eta_m - 1}$$

where $B(\eta)$ is a normalizer and $\boldsymbol{\eta}$ is M-dimensional, $\eta_m > 0$

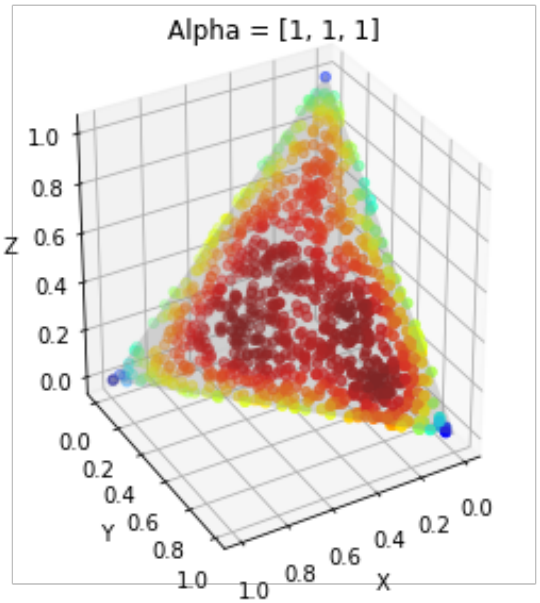


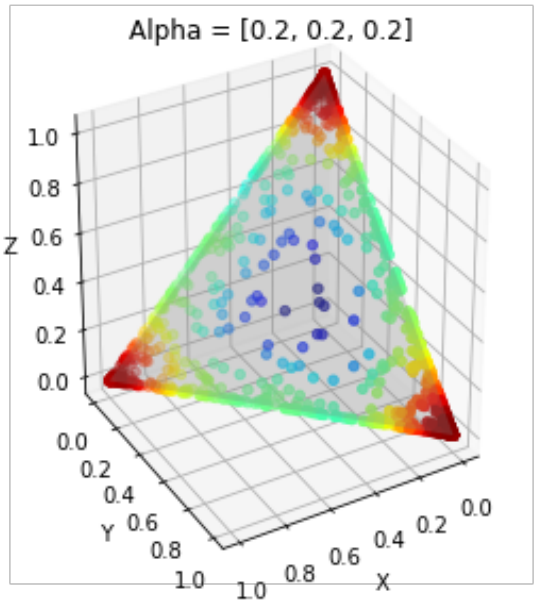
- Defined over the (M-1)-dimensional *simplex*:

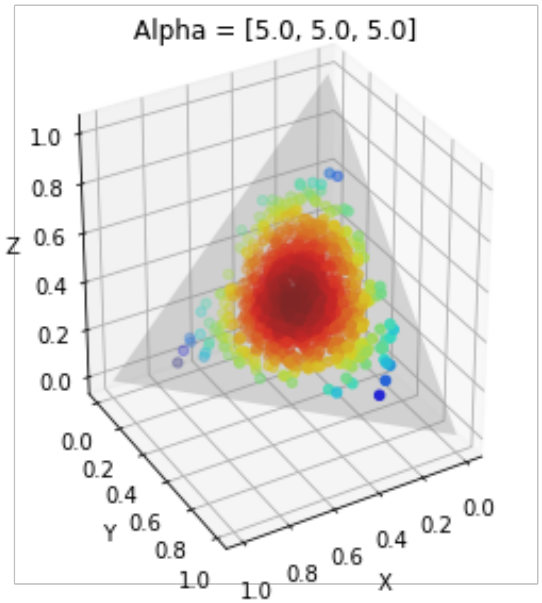
$$0 < \beta_1, \ldots, \beta_M < 1 \qquad \beta_1 + \ldots + \beta_M = 1$$

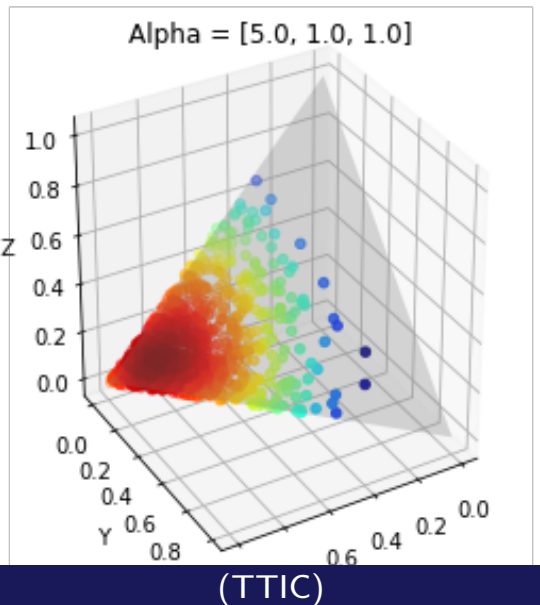


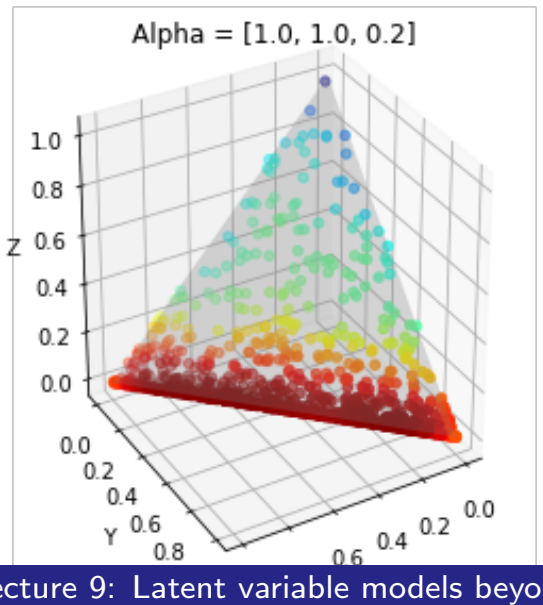
Dirichlet distribution: Visualization

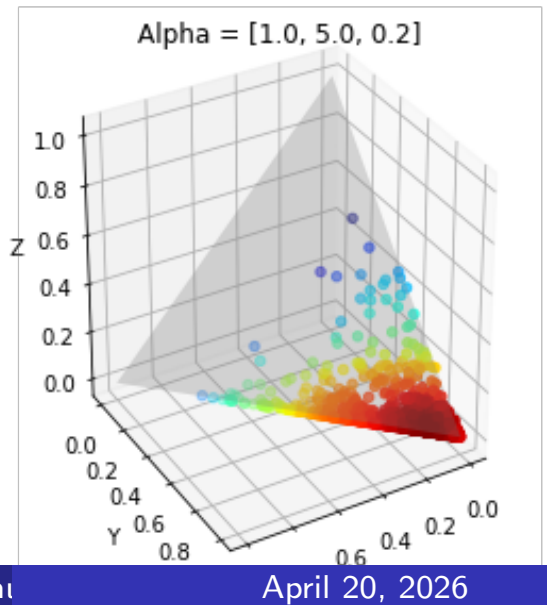






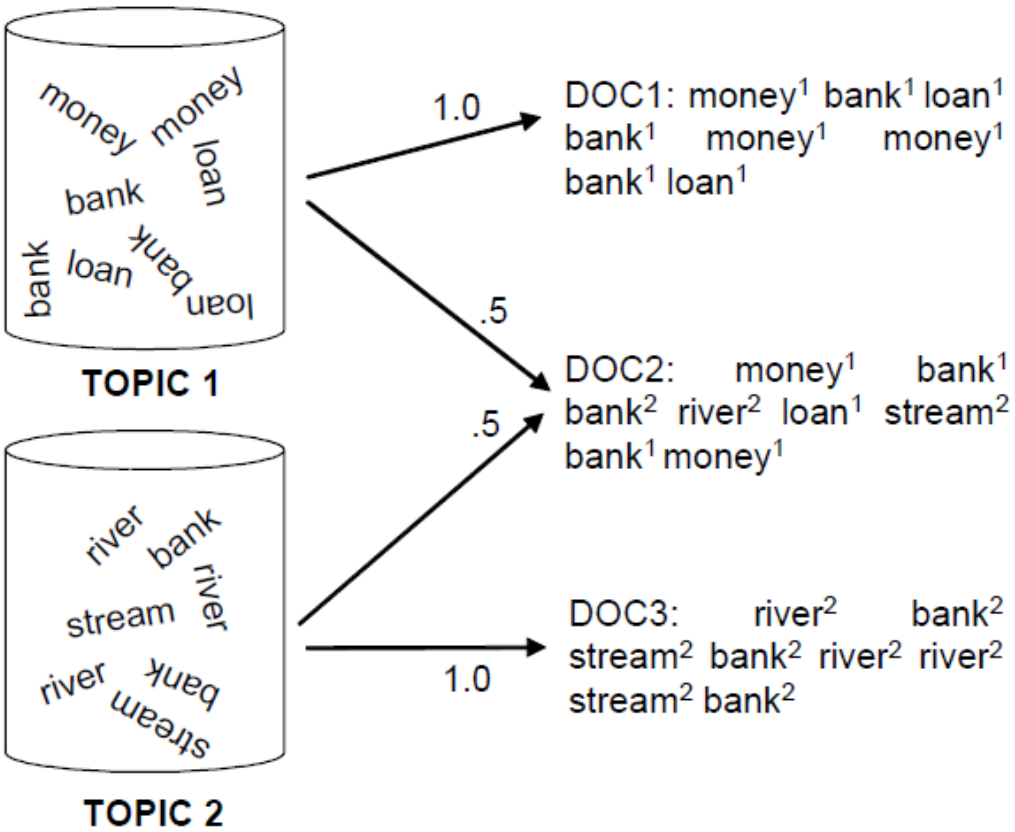






Mixture of topics

- A more realistic assumption: documents are on *mixtures* of topics
- Each topic t has its own distribution over words $\boldsymbol{\beta}_t$
- A document d has its own distribution over T topics $\boldsymbol{\theta}_d$



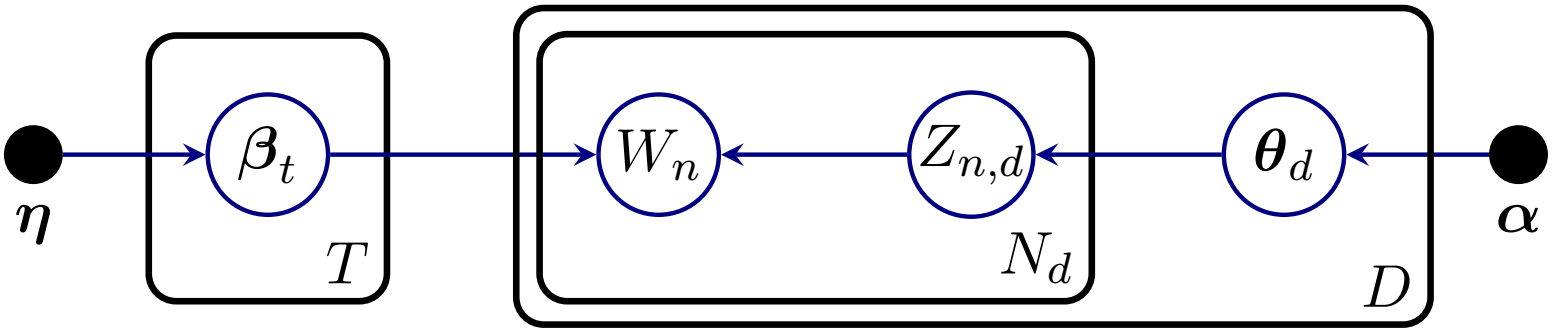
$$\boldsymbol{\theta}_1 = [1, 0]$$
$$\boldsymbol{\theta}_2 = [.5, .5]$$
$$\boldsymbol{\theta}_3 = [0, 1]$$

[source: Steyvers & Griffiths]

Topic model: generative process

[Griffiths & Steyvers 2002–2004, Blei et al. 2002–2003]

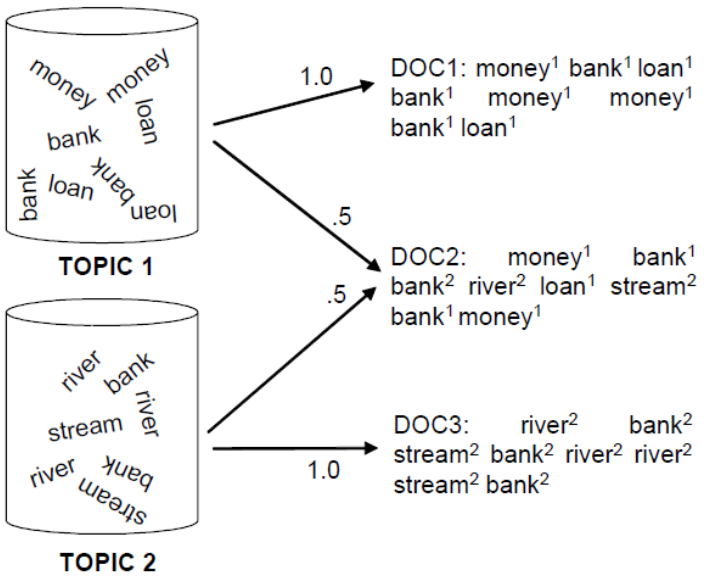
- D documents, document d with N_d words
- Model probability over topics for a document $\boldsymbol{\theta}_d$ with Dirichlet



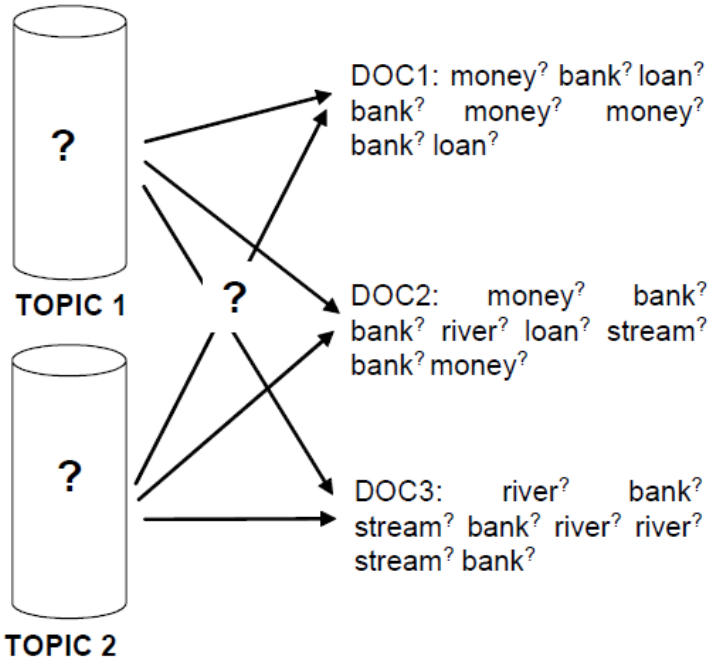
- Recipe for generating a document with N words:
  (1) generate the set of T topics: draw each from Dirichlet($\boldsymbol{\eta}$)
  (2) for document d, generate $\boldsymbol{\theta}_d$: draw from Dirichlet($\boldsymbol{\alpha}$)
  (3) For each $n = 1, \ldots, N_d$ draw topic $Z_n \in \{0, \ldots, T\}$ from multinomial($\boldsymbol{\theta}$); then draw word W_n from multinomial($\boldsymbol{\beta}_{Z_n}$)
- Also referred to as (smoothed) latent Dirichlet allocation (LDA)

Learning topic models

generative model



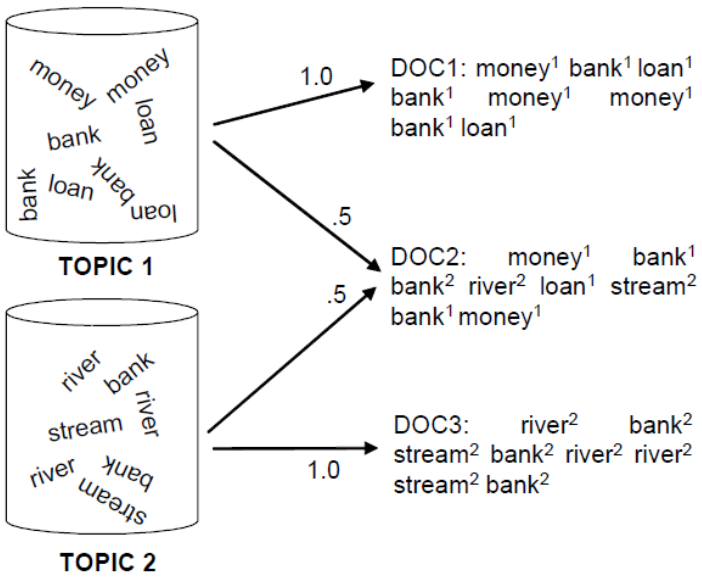
estimation from data



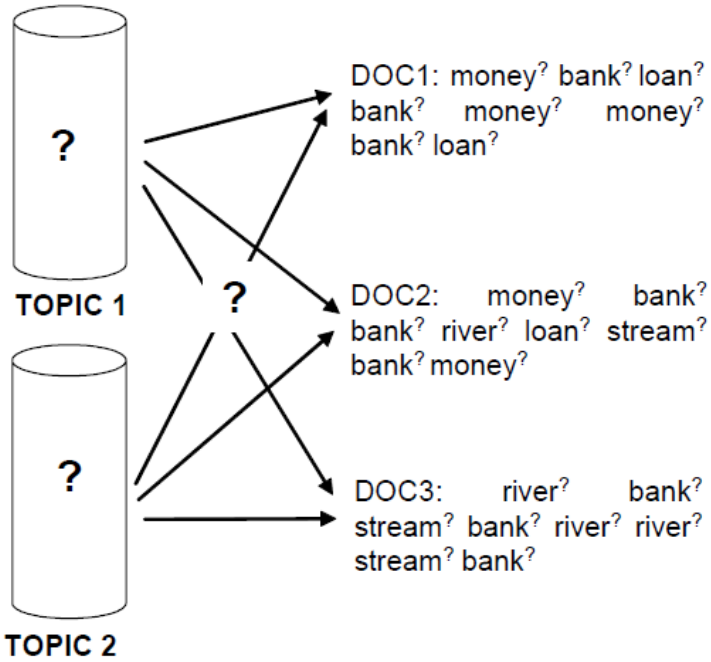
- Common situation in learning: model with *latent* (hidden) variables
- If we knew $Z_{d,n}$ we could estimate easily (maximum likelihood)
- In reality, need to estimate everything: Zs, $\boldsymbol{\theta}$s, $\boldsymbol{\beta}$s.

Learning topic models

generative model



estimation from data



- Common situation in learning: model with *latent* (hidden) variables
- If we knew $Z_{d,n}$ we could estimate easily (maximum likelihood)
- In reality, need to estimate everything: Zs, $\boldsymbol{\theta}$s, $\boldsymbol{\beta}$s.
- Multiple ways; e.g., Expectation Maximization (EM) algorithm and sampling-based methods (MCMC and Gibbs sampling)

Example topics

- From Steyvers & Griffiths, "Probabilistic Topic Models", 2007
- Touchstone Applied Science Associates (TASA) corpus: 37,000 text passages
- Example topics (out of 300):

Topic 247

word	prob.
DRUGS	.069
DRUG	.060
MEDICINE	.027
EFFECTS	.026
BODY	.023
MEDICINES	.019
PAIN	.016
PERSON	.016
MARIJUANA	.014
LABEL	.012
ALCOHOL	.012
DANGEROUS	.011
ABUSE	.009
EFFECT	.009
KNOWN	.008
PILLS	.008

Topic 5

word	prob.
RED	.202
BLUE	.099
GREEN	.096
YELLOW	.073
WHITE	.048
COLOR	.048
BRIGHT	.030
COLORS	.029
ORANGE	.027
BROWN	.027
PINK	.017
LOOK	.017
BLACK	.016
PURPLE	.015
CROSS	.011
COLORED	.009

Topic 43

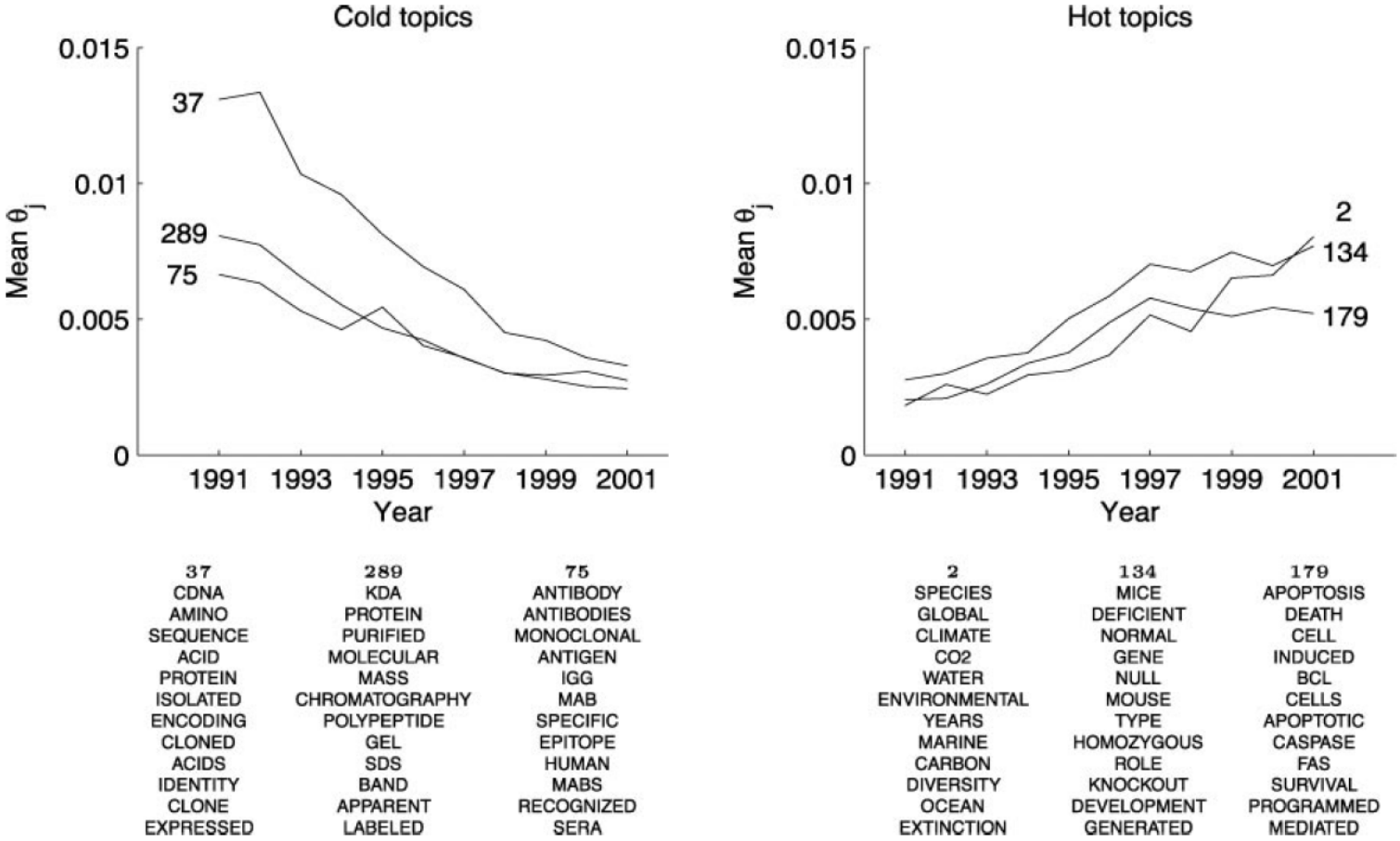
word	prob.
MIND	.081
THOUGHT	.066
REMEMBER	.064
MEMORY	.037
THINKING	.030
PROFESSOR	.028
FELT	.025
REMEMBERED	.022
THOUGHTS	.020
FORGOTTEN	.020
MOMENT	.020
THINK	.019
THING	.016
WONDER	.014
FORGET	.012
RECALL	.012

Topic 56

word	prob.
DOCTOR	.074
DR.	.063
PATIENT	.061
HOSPITAL	.049
CARE	.046
MEDICAL	.042
NURSE	.031
PATIENTS	.029
DOCTORS	.028
HEALTH	.025
MEDICINE	.017
NURSING	.017
DENTAL	.015
NURSES	.013
PHYSICIAN	.012
HOSPITALS	.011

Application: Tracking topics in scientific literature

- From Griffiths & Steyvers, "Finding Scientific Topics", 2004



- Note: LDA/topic models was a very hot topic in ML in 2000-2010, now not; but still can be useful

Language models

Density estimation for sequences

Last few lectures: Clustering, mixture models, topic models

- Divide up a data set into groups (clusters)
- Or, into a mixture of groups (components, topics)
- Can we take into account the temporal structure of data points in the data?
- E.g. words in a document, daily weather conditions, stock prices over time, ...

Density estimation for sequences

Last few lectures: Clustering, mixture models, topic models

- Divide up a data set into groups (clusters)
- Or, into a mixture of groups (components, topics)
- Can we take into account the temporal structure of data points in the data?
- E.g. words in a document, daily weather conditions, stock prices over time, ...

Next:

- Modeling discrete sequences with language models

Language models

Consider sequences of discrete items; call them "words"

A language model is a distribution over words, $p(\mathbf{w})$ where $\mathbf{w} = w_1, w_2, ..., w_K$

(Note: There is a **bit** of disagreement about this definition)

Language models

Consider sequences of discrete items; call them "words"

A language model is a distribution over words, $p(\mathbf{w})$ where $\mathbf{w} = w_1, w_2, ..., w_K$

(Note: There is a **bit** of disagreement about this definition)

Applications:

- Speech recognition
- Spelling correction
- Machine translation
- Natural language generation
- ...
- Almost any problem that requires predicting or generating a sequence!

Why are language models needed?

Why are language models needed?

- In speech recognition, to disambiguate acoustically similar utterances using prior knowledge about word sequences
  - $p(\text{“It is easy to recognize speech”}) = .0001$

Why are language models needed?

- In speech recognition, to disambiguate acoustically similar utterances using prior knowledge about word sequences
  - $p(\text{“It is easy to recognize speech”}) = .0001$
  - $p(\text{“It is easy to wreck a nice beach”}) = .00000001$

Why are language models needed?

- In speech recognition, to disambiguate acoustically similar utterances using prior knowledge about word sequences
  - $p(\text{“It is easy to recognize speech”}) = .0001$
  - $p(\text{“It is easy to wreck a nice beach”}) = .00000001$
- In machine translation, to choose the most natural sequence of words in the target language that represents the “chunks” of meaning in the source language

Why are language models needed?

- In speech recognition, to disambiguate acoustically similar utterances using prior knowledge about word sequences
  - $p(\text{“It is easy to recognize speech”}) = .0001$
  - $p(\text{“It is easy to wreck a nice beach”}) = .00000001$
- In machine translation, to choose the most natural sequence of words in the target language that represents the “chunks” of meaning in the source language
- For general natural language tasks: To generate a continuation given a prompt
- (And any task can be described as generating a continuation)

History-based statistical LMs

- $p(\mathbf{w})$ can be expanded using the chain rule:

$$p(\mathbf{w}) = \prod_{i=1}^{K} p(w_i|w_1, \ldots, w_{i-1}) = \prod_{i=1}^{K} p(w_i|h_i)$$

  where $h_i = (w_1, \ldots, w_{i-1})$ is the *history* for word w_i.

- Note: First & last words typically assumed to be a sentence boundary marker, $w_1 = w_K =<>$

History-based statistical LMs

- $p(\mathbf{w})$ can be expanded using the chain rule:

$$p(\mathbf{w}) = \prod_{i=1}^{K} p(w_i|w_1, \ldots, w_{i-1}) = \prod_{i=1}^{K} p(w_i|h_i)$$

  where $h_i = (w_1, \ldots, w_{i-1})$ is the *history* for word w_i.
- Note: First & last words typically assumed to be a sentence boundary marker, $w_1 = w_K =<>$
- Too many possible histories $\Rightarrow$ reduce to equivalence classes $\phi(h_i)$, such that $p(w_i|h_i) \approx p(w_i|\phi(h_i))$

History-based statistical LMs

- $p(\mathbf{w})$ can be expanded using the chain rule:

$$p(\mathbf{w}) = \prod_{i=1}^{K} p(w_i|w_1, \ldots, w_{i-1}) = \prod_{i=1}^{K} p(w_i|h_i)$$

  where $h_i = (w_1, \ldots, w_{i-1})$ is the *history* for word w_i.
- Note: First & last words typically assumed to be a sentence boundary marker, $w_1 = w_K =<>$
- Too many possible histories $\Rightarrow$ reduce to equivalence classes $\phi(h_i)$, such that $p(w_i|h_i) \approx p(w_i|\phi(h_i))$
- Good equivalence classes maximize the information about the current word given the class $\phi(h_i)$

History-based statistical LMs

- $p(\mathbf{w})$ can be expanded using the chain rule:

$$p(\mathbf{w}) = \prod_{i=1}^{K} p(w_i|w_1, \ldots, w_{i-1}) = \prod_{i=1}^{K} p(w_i|h_i)$$

  where $h_i = (w_1, \ldots, w_{i-1})$ is the *history* for word w_i.
- Note: First & last words typically assumed to be a sentence boundary marker, $w_1 = w_K =<>$
- Too many possible histories $\Rightarrow$ reduce to equivalence classes $\phi(h_i)$, such that $p(w_i|h_i) \approx p(w_i|\phi(h_i))$
- Good equivalence classes maximize the information about the current word given the class $\phi(h_i)$
- (LMs requiring the full word sequence $\mathbf{w}$ can be used, but usually in a "rescoring" setting – more later...)

n-gram language models

- In *n*-gram LMs, the history equivalence class is the previous $n-1$ words: $\phi(h_i) = (w_{i-1}, \ldots, w_{i-n+1})$
- For example:
  - bigram LM $p(w_i|w_{i-1})$
  - trigram LM $p(w_i|w_{i-1}, w_{i-2})$
- Trigrams were for a long time the dominant LM in large-vocabulary recognition research, but longer histories now being used with large training sets (even arbitrarily long histories)

n-gram example

Consider the sentence:
$\mathbf{w}$ = "*The quick brown fox jumped over the lazy dog.*"

$$
\begin{aligned}
p(w_1, \ldots, w_n) \quad = \quad & p(\text{the}|<>) \\
& p(\text{quick}|\text{the}, <>) \\
& p(\text{brown}|\text{quick, the}) \\
& \ldots \\
& p(\text{dog}|\text{lazy, the}) \\
& p(<> |\text{dog, lazy})
\end{aligned}
$$

Where do the probabilities come from?

Maximum-likelihood estimate of *n*-gram probabilities given some training set of text:

$$\hat{p}(\text{quick}|\ <>, \text{the}) = \frac{\text{count}(<>,\text{the, quick})}{\text{count}(<>,\text{the})}$$

Trigram example (Jelinek '97)

1	The	are	to	know	the	issues	necessary
2	This	will		have	this	problems	data
3	One	the		understand	these	the	information
4	Two	would		do	problems		above
5	A	also		get	any		other
6	Three	do		the	a		time
7	Please	need		use	problem		people
8	In			provide	them		operators
9	We			insert	all		tools
•				•			•
•				•			•
96				write			jobs
97				me			MVS
98				resolve			old
•							•
•							•
1639							reception
1640							shop
1641							important

Trigram example (2)

1	role	and	the	next	be	metting	of
2	thing	from			two	months	<>
3	that	in				years	
4	to	to				meetings	
5	contact	are				to	
6	parts	with				weeks	
7	point	were				days	
8	for	requiring					
9	issues	still					
•		•					
•		•					
61		being					
62		during					
63		I					
64		involved					
65		would					
66		within					

Random sentence generation example: Air travel domain bigram

Show me the flight earliest flight from Denver
How many flights that flight leaves around is the Eastern Denver
I want a first class
Show me a reservation the last flight from Baltimore for the first
I would like to fly from Dallas
I get from Pittsburgh
Which just small
In Denver on October
I would like to San Francisco
Is flight flying
What flights from Boston to San Francisco
How long can you book a hundred dollars
I would like to Denver to Boston and Boston
Make ground transportation is the cheapest
Are the next week on AA eleven ten
First class
How many airlines from Boston on May thirtieth
What is the city of three PM
What about twelve and Baltimore

Random sentence generation example: Air travel domain trigram

What type of aircraft
What is the fare on flight two seventy two
Show me the flights I've Boston to San Francisco on Monday
What is the cheapest one way
Okay on flight number seven thirty six
What airline leaves earliest
Which airlines from Philadelphia to Dallas
I'd like to leave at nine eight
What airline
How much does it cost
How many stops does Delta flight five eleven o'clock PM that go from
What AM
Is Eastern from Denver before noon
Earliest flight from Dallas
I need to Philadelphia
Describe to Baltimore on Wednesday from Boston
I'd like to depart before five o'clock PM
Which flights do these flights leave after four PM and lunch and <unknown>

Evaluating LMs: Cross-entropy

- If X is a discrete random variable taking one of N values with probabilities $p_1, \ldots, p_N$, respectively, then the entropy of X is

$$H(X) = -\sum_{i=1}^{N} p_i \log_2 p_i$$

Evaluating LMs: Cross-entropy

- If X is a discrete random variable taking one of N values with probabilities $p_1, \ldots, p_N$, respectively, then the entropy of X is
$$H(X) = -\sum_{i=1}^{N} p_i \log_2 p_i$$
- The *cross-entropy* of a model $\hat{p}(X)$ with respect to some (i.i.d.) data set $\mathbf{X} = \{x_1, \ldots, x_n\}$ is $H_{\hat{p}}(\mathbf{X}) = -\frac{1}{n} \log_2 \hat{p}(\mathbf{X})$

Evaluating LMs: Cross-entropy

- If X is a discrete random variable taking one of N values with probabilities $p_1, \ldots, p_N$, respectively, then the entropy of X is

$$H(X) = -\sum_{i=1}^{N} p_i \log_2 p_i$$

- The *cross-entropy* of a model $\hat{p}(X)$ with respect to some (i.i.d.) data set $\mathbf{X} = \{x_1, \ldots, x_n\}$ is $H_{\hat{p}}(\mathbf{X}) = -\frac{1}{n} \log_2 \hat{p}(\mathbf{X})$
- For an autoregressive LM $\hat{p}(\cdot)$ on a test set $\mathbf{w} = (w_1, \ldots, w_n)$, w_i are not i.i.d. but they are i.i.d. given their histories:

$$\begin{aligned} H_{\hat{p}}(\mathbf{w}) &= -\frac{1}{n} \log_2 \hat{p}(\mathbf{w}) \\ &= -\frac{1}{n} \log_2 \prod_{i=1}^{n} \hat{p}(w_i | \phi(h_i)) \\ &= -\frac{1}{n} \sum_{i=1}^{n} \log_2 \hat{p}(w_i | \phi(h_i)) \end{aligned}$$

Evaluating LMs: Cross-entropy (2)

- Intuition: This is the number of bits per word needed to encode this data set using the model
- (This intuition can be made more precise with a bit more information theory; we won't do that here)
- For English texts, cross-entropy ranges from around 6 to 10 bits/word for n-gram language models

Evaluating LMs: Cross-entropy (2)

- Intuition: This is the number of bits per word needed to encode this data set using the model
- (This intuition can be made more precise with a bit more information theory; we won't do that here)
- For English texts, cross-entropy ranges from around 6 to 10 bits/word for n-gram language models
- (Note: You can also use a different base for the log. E.g., if using natural log, then the units of entropy/cross-entropy are nats)

Evaluating LMs: Perplexity

- Perplexity is related to the cross-entropy via

$$PP_p(\mathbf{X}) = 2^{H_p(\mathbf{X})} = 2^{-\frac{1}{n}\log_2 \hat{p}(\mathbf{X})} = \left(\frac{1}{\hat{p}(\mathbf{X})}\right)^{1/n}$$

Evaluating LMs: Perplexity

- Perplexity is related to the cross-entropy via

$$PP_p(\mathbf{X}) = 2^{H_p(\mathbf{X})} = 2^{-\frac{1}{n}\log_2 \hat{p}(\mathbf{X})} = \left(\frac{1}{\hat{p}(\mathbf{X})}\right)^{1/n}$$

- Given a test set $\mathbf{w} = (w_1, \ldots, w_n)$ and an autoregressive language model $\hat{p}(w_i|\phi(h_i))$,

$$\begin{aligned} PP_p(\mathbf{w}) &= 2^{-\frac{1}{n}\sum_{i=1}^{n}\log_2 \hat{p}(w_i|\phi(h_i))} \\ &= \left(\prod_{i=1}^{n}\frac{1}{\hat{p}(w_i|\phi(h_i))}\right)^{1/n} \end{aligned}$$

Evaluating LMs: Perplexity

- Perplexity is related to the cross-entropy via

$$PP_p(\mathbf{X}) = 2^{H_p(\mathbf{X})} = 2^{-\frac{1}{n}\log_2 \hat{p}(\mathbf{X})} = \left(\frac{1}{\hat{p}(\mathbf{X})}\right)^{1/n}$$

- Given a test set $\mathbf{w} = (w_1, \ldots, w_n)$ and an autoregressive language model $\hat{p}(w_i|\phi(h_i))$,

$$\begin{aligned} PP_p(\mathbf{w}) &= 2^{-\frac{1}{n}\sum_{i=1}^{n}\log_2 \hat{p}(w_i|\phi(h_i))} \\ &= \left(\prod_{i=1}^{n}\frac{1}{\hat{p}(w_i|\phi(h_i))}\right)^{1/n} \end{aligned}$$

- So perplexity is the *geometric mean of the per-word inverse probability*

Evaluating LMs: Perplexity

- Perplexity is related to the cross-entropy via

$$PP_p(\mathbf{X}) = 2^{H_p(\mathbf{X})} = 2^{-\frac{1}{n}\log_2 \hat{p}(\mathbf{X})} = \left(\frac{1}{\hat{p}(\mathbf{X})}\right)^{1/n}$$

- Given a test set $\mathbf{w} = (w_1, \ldots, w_n)$ and an autoregressive language model $\hat{p}(w_i|\phi(h_i))$,

$$\begin{aligned} PP_p(\mathbf{w}) &= 2^{-\frac{1}{n}\sum_{i=1}^{n}\log_2 \hat{p}(w_i|\phi(h_i))} \\ &= \left(\prod_{i=1}^{n}\frac{1}{\hat{p}(w_i|\phi(h_i))}\right)^{1/n} \end{aligned}$$

- So perplexity is the *geometric mean of the per-word inverse probability*
- Lower entropy/perplexity $\Rightarrow$ lower uncertainty about the following word $\Rightarrow$ better language model

Evaluating LMs: Perplexity (4)

- For English text, per-word perplexity ranges from around 25 to several 100 for n-gram language models.
- Note: We can also define language models and perplexity per character or per token for any other definition of token

Perplexity of a distribution vs. perplexity as an evaluation metric

- We have talked about *perplexity of a model evaluated on a test set*
- Another notion is *perplexity of a distribution*

Perplexity of a distribution vs. perplexity as an evaluation metric

- We have talked about *perplexity of a model evaluated on a test set*
- Another notion is *perplexity of a distribution*
- If X is a discrete random variable taking one of N values with probabilities $p_1, \ldots, p_N$, respectively, then the entropy of X is

$$H(X) = -\sum_{i=1}^{N} p_i \log_2 p_i$$

- The perplexity of X is $PP(X) = 2^{H(X)}$

Perplexity of a distribution vs. perplexity as an evaluation metric

- We have talked about *perplexity of a model evaluated on a test set*
- Another notion is *perplexity of a distribution*
- If X is a discrete random variable taking one of N values with probabilities $p_1, \ldots, p_N$, respectively, then the entropy of X is

$$H(X) = -\sum_{i=1}^{N} p_i \log_2 p_i$$

- The perplexity of X is $PP(X) = 2^{H(X)}$
- Intuition: Perplexity of an LM, viewed as a distribution, is the average number of words/tokens possible after a given history (the average *branching factor* of the LM)

Perplexity of a distribution vs. perplexity as an evaluation metric

- We have talked about *perplexity of a model evaluated on a test set*
- Another notion is *perplexity of a distribution*
- If X is a discrete random variable taking one of N values with probabilities $p_1, \ldots, p_N$, respectively, then the entropy of X is

$$H(X) = -\sum_{i=1}^{N} p_i \log_2 p_i$$

- The perplexity of X is $PP(X) = 2^{H(X)}$
- Intuition: Perplexity of an LM, viewed as a distribution, is the average number of words/tokens possible after a given history (the average *branching factor* of the LM)
- What is the perplexity of a uniform LM? What is the minimum perplexity possible?

Problems with (basic) n-grams...

- Can't incorporate long-distance relationships
- Hard to deal with unseen n-grams
- Do not capture meaning for more complex tasks like sentence understanding and generation

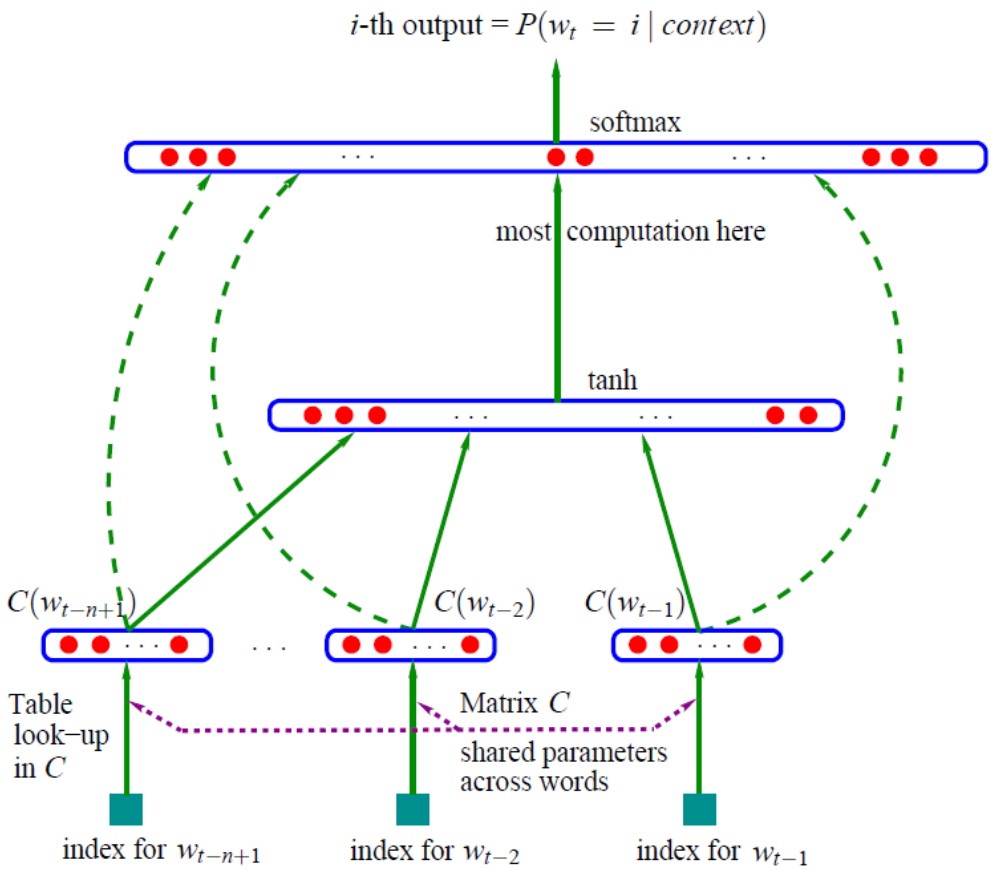
Neural network language models

- Model $p(w_i|h(w_i))$ with a neural network
- Inputs: one-hot vectors for context words
- Outputs: posterior probability of each next word

Neural network language models: Feedforward

Neural n-gram models

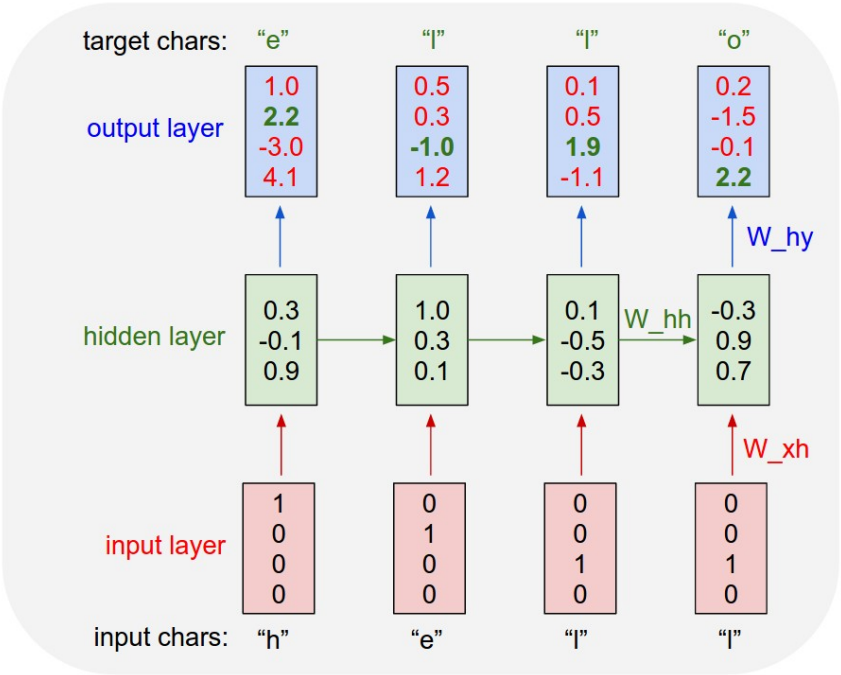
- Introduced by Bengio *et al.* in 2003
- Different way of dealing with rare/unseen n-grams



[Bengio+ 2003]

Neural network language models: Recurrent

- Very large (in principle) context needed to predict next word
- Example: The dog who was chasing the squirrel around the tree got tired.
- RNN LMs allow us to model arbitrarily long contexts
- Introduced by Mikolov *et al.* in 2010



[Karpathy]